Repaso de SQL

Bases de Datos 2 — guía ampliada de ejercicios y dudas

Contenido: SQL relacional y SQL, operadores, NULL, CASE, DECODE, COALESCE, JOIN, COUNT, subconsultas, consultas multicolumna, CTE recursivas, GROUP BY, HAVING y funciones de ventana.

1. Relacional y no relacional

Base de datos relacional: organiza la información en tablas de filas y columnas y relaciona las tablas mediante claves. SQL es el lenguaje habitual para consultar estos datos.

NoSQL: no depende obligatoriamente de tablas relacionadas. Puede trabajar con documentos, clave-valor, grafos o columnas. MongoDB es un ejemplo de base documental.

Idea para recordar: relacional = tablas + relaciones + SQL. NoSQL = modelos más flexibles, por ejemplo documentos.

2. Operadores y filtros

BETWEEN

Ejercicio: traer empleados cuyo salario esté entre 10.000 y 50.000. BETWEEN incluye ambos extremos.

```
SELECT *
FROM HR.EMPLOYEES
WHERE SALARY BETWEEN 10000 AND 50000;
```

Equivale a: SALARY >= 10000 AND SALARY <= 50000.

IN

```
SELECT *
FROM HR.EMPLOYEES
WHERE DEPARTMENT_ID IN (80, 90, 15);
```

Es equivalente a usar tres condiciones unidas con OR, pero resulta más corto y legible.

LIKE

```
SELECT FIRST_NAME
FROM HR.EMPLOYEES
WHERE FIRST_NAME LIKE 'And%';

SELECT FIRST_NAME
FROM HR.EMPLOYEES
WHERE FIRST_NAME NOT LIKE 'And%';
```

% = cero o más caracteres. _ = exactamente un carácter. Por eso 'And%' encuentra nombres que comienzan por And.

LIKE con _

```
SELECT FIRST_NAME
FROM HR.EMPLOYEES
WHERE FIRST_NAME LIKE '_a%';
```

El primer carácter puede ser cualquiera; el segundo debe ser a; después puede haber cualquier cantidad de caracteres.

IS NULL

```
SELECT LAST_NAME, COMMISSION_PCT
FROM HR.EMPLOYEES
WHERE COMMISSION_PCT IS NULL;
```

Para NULL se utiliza IS NULL o IS NOT NULL. No se debe escribir COMMISSION_PCT = NULL.

3. NULL y lógica de tres valores

NULL representa ausencia, desconocimiento o falta de un valor. No es lo mismo que 0. En SQL, NULL = NULL no produce TRUE: produce UNKNOWN.

```
SELECT CASE
           WHEN NULL = NULL THEN 'TRUE'
           WHEN NULL <> NULL THEN 'FALSE'
           ELSE 'NI UNO NI OTRO'
       END AS RESULTADO
FROM DUAL;
```

El resultado es NI UNO NI OTRO. Esta es una de las razones por las que se usa IS NULL.

Regla: ¿quiero encontrar NULL? → IS NULL. ¿Quiero excluir NULL? → IS NOT NULL.

4. DUAL

DUAL es una tabla especial de Oracle que permite ejecutar expresiones sin consultar una tabla de datos concreta.

```
SELECT *
FROM DUAL;

SELECT 5 + 6 + 7 + 8 + 9 / 2 * 3 AS RESULTADO
FROM DUAL;
```

En una expresión aritmética, multiplicación y división se realizan antes que suma y resta, salvo que los paréntesis cambien el orden.

5. CASE, DECODE y COALESCE

CASE

CASE permite tomar decisiones y es parecido a IF / ELSE IF / ELSE.

```
SELECT LAST_NAME,
       SALARY,
       CASE
           WHEN SALARY < 5000 THEN 'BAJO'
           WHEN SALARY BETWEEN 5000 AND 10000 THEN 'MEDIO'
           ELSE 'ALTO'
       END AS CATEGORIA
FROM HR.EMPLOYEES;
```

Estructura: CASE → WHEN condición → THEN resultado → ELSE resultado → END.

DECODE

```
SELECT LAST_NAME,
       DECODE(DEPARTMENT_ID,
              10, 'Administración',
              20, 'Marketing',
              30, 'Compras',
              'Otro') AS DEPARTAMENTO
FROM HR.EMPLOYEES;
```

DECODE es útil para equivalencias exactas. CASE es más flexible cuando necesitamos condiciones como >, < o BETWEEN.

COALESCE

```
SELECT LAST_NAME,
       COALESCE(TO_CHAR(COMMISSION_PCT), 'SIN COMISION') AS COMISION
FROM HR.EMPLOYEES;
```

COALESCE devuelve el primer valor que no sea NULL. Por ejemplo, COALESCE(NULL, NULL, 'Hola', 'Adiós') devuelve Hola.

6. LEFT JOIN: ON vs WHERE

Ejercicio: mostrar los departamentos que no tienen empleados.

```
SELECT D.DEPARTMENT_NAME
FROM HR.DEPARTMENTS D
LEFT JOIN HR.EMPLOYEES E
  ON E.DEPARTMENT_ID = D.DEPARTMENT_ID
WHERE E.EMPLOYEE_ID IS NULL;
```

LEFT JOIN conserva todos los departamentos. Si no existe empleado relacionado, las columnas de E quedan en NULL.

Patrón importante: LEFT JOIN + WHERE tabla_derecha.clave IS NULL → buscar registros de la izquierda sin coincidencia.

La diferencia que hay que entender

```
SELECT D.DEPARTMENT_NAME, E.LAST_NAME
FROM HR.DEPARTMENTS D
LEFT JOIN HR.EMPLOYEES E
  ON E.DEPARTMENT_ID = D.DEPARTMENT_ID
WHERE E.SALARY > 5000;

SELECT D.DEPARTMENT_NAME, E.LAST_NAME
FROM HR.DEPARTMENTS D
LEFT JOIN HR.EMPLOYEES E
  ON E.DEPARTMENT_ID = D.DEPARTMENT_ID
 AND E.SALARY > 5000;
```

Primera: WHERE filtra el resultado final. Los empleados con salario NULL o <= 5000 desaparecen; incluso los departamentos sin empleados no pasan el WHERE.

Segunda: la condición de salario está en ON. Todos los departamentos se conservan, pero solo se relacionan empleados con salario > 5000.

Resumen: ON decide coincidencias del JOIN; WHERE filtra las filas resultantes.

7. COUNT, AVG y SUM con NULL

```
SELECT COUNT(*) AS FILAS,
       COUNT(COMMISSION_PCT) AS CON_COMISION,
       AVG(COMMISSION_PCT) AS PROM_REAL
FROM HR.EMPLOYEES;
```

COUNT(*) cuenta filas completas. Una fila se cuenta aunque COMMISSION_PCT sea NULL.

COUNT(COMMISSION_PCT) cuenta únicamente valores no NULL.

AVG(COMMISSION_PCT) ignora los NULL y promedia solo los valores existentes.

```
SELECT SUM(COMMISSION_PCT) AS SUMA_COMISIONES
FROM HR.EMPLOYEES;
```

SUM también ignora NULL. Si todos los valores son NULL, el resultado puede ser NULL.

Ejemplo importante: si hay 107 empleados y solo 35 tienen comisión, COUNT(*) = 107, COUNT(COMMISSION_PCT) = 35. Los 72 NULL sí cuentan para COUNT(*), pero no para

COUNT(COMMISSION_PCT).

8. Subconsulta correlacionada

Ejercicio: mostrar el apellido de cada empleado y cuántas personas hay en su mismo departamento.

```
SELECT E.LAST_NAME,
       (
           SELECT COUNT(*)
           FROM HR.EMPLOYEES C
           WHERE C.DEPARTMENT_ID = E.DEPARTMENT_ID
       ) AS COMPANEROS
FROM HR.EMPLOYEES E;
```

La subconsulta depende de E.DEPARTMENT_ID. Por cada empleado externo, cuenta empleados que tienen el mismo departamento.

¿Se cuenta a sí mismo? Sí. Si hay 5 empleados en el departamento, cada uno obtiene 5. Para excluirse: AND C.EMPLOYEE_ID <> E.EMPLOYEE_ID.

9. IN + subconsulta + GROUP BY

```
SELECT E.LAST_NAME,
       COUNT(*) AS CANTIDAD
FROM HR.EMPLOYEES E
WHERE E.DEPARTMENT_ID IN (
    SELECT DEPARTMENT_ID
    FROM HR.EMPLOYEES
    WHERE LAST_NAME = 'King'
)
GROUP BY E.LAST_NAME;
```

Primero se obtienen los departamentos donde existe King. Después se buscan empleados de esos departamentos y finalmente se agrupan por apellido.

10. Consulta multicolumna

```
SELECT LAST_NAME,
       JOB_ID,
       MANAGER_ID
FROM HR.EMPLOYEES
WHERE (JOB_ID, MANAGER_ID) IN (
    SELECT JOB_ID,
           MANAGER_ID
    FROM HR.EMPLOYEES
    WHERE DEPARTMENT_ID = 90
);
```

Aquí se compara una pareja de valores: JOB_ID y MANAGER_ID. Una fila coincide si la combinación completa aparece en la subconsulta.

Si alguna parte de la comparación involucra NULL, la igualdad puede resultar UNKNOWN. Por eso estas consultas no deben interpretarse como si NULL fuera un valor normal.

11. CTE recursiva: JERARQUÍA

¿Es una sentencia recursiva? Sí. La CTE JERARQUIA se referencia a sí misma en la segunda parte del UNION ALL. Esto permite recorrer la relación empleado → manager por niveles.

```
WITH JERARQUIA (EMPLOYEE_ID, LAST_NAME, MANAGER_ID, NIVEL) AS (
    SELECT EMPLOYEE_ID,
           LAST_NAME,
           MANAGER_ID,
           1
    FROM HR.EMPLOYEES
    WHERE MANAGER_ID IS NULL
```

```
    UNION ALL

    SELECT E.EMPLOYEE_ID,
           E.LAST_NAME,
           E.MANAGER_ID,
           J.NIVEL + 1
    FROM HR.EMPLOYEES E
    JOIN JERARQUIA J
      ON E.MANAGER_ID = J.EMPLOYEE_ID
)
CYCLE EMPLOYEE_ID SET ES_CICLO TO 'Y' DEFAULT 'N'
SELECT LAST_NAME,
       NIVEL,
       ES_CICLO
FROM JERARQUIA;
```

Paso 1: el caso base encuentra empleados sin manager y les asigna NIVEL = 1.

Paso 2: la parte recursiva busca empleados cuyo MANAGER_ID coincide con un EMPLOYEE_ID ya encontrado.

Paso 3: aumenta NIVEL en 1 y repite.

UNION ALL: une el caso base con los resultados de cada nivel.

CYCLE: permite detectar ciclos para evitar recorridos problemáticos en datos jerárquicos.

12. GROUP BY + HAVING + OVER()

Ejercicio: calcular la nómina por departamento y, además, mostrar la nómina total.

```
SELECT DEPARTMENT_ID,
       SUM(SALARY) AS NOMINA,
       SUM(SUM(SALARY)) OVER () AS NOMINA_TOTAL
FROM HR.EMPLOYEES
GROUP BY DEPARTMENT_ID;
```

El SUM interno calcula la nómina de cada departamento. El SUM externo es una función de ventana y suma las nóminas resultantes.

HAVING

```
SELECT DEPARTMENT_ID,
       SUM(SALARY) AS NOMINA,
       SUM(SUM(SALARY)) OVER () AS NOMINA_TOTAL
FROM HR.EMPLOYEES
GROUP BY DEPARTMENT_ID
HAVING SUM(SALARY) > 10000;
```

Se usa HAVING porque estamos filtrando por una función de agregación. WHERE filtra filas antes de GROUP BY; HAVING filtra grupos después de GROUP BY.

Idea: WHERE → filas. GROUP BY → grupos. HAVING → grupos ya calculados.

13. OVER(): ROWS y RANGE

```
SELECT LAST_NAME,
       SALARY,
       SUM(SALARY) OVER (
           ORDER BY SALARY
           ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
       ) AS ACUM_ROWS,
       SUM(SALARY) OVER (
           ORDER BY SALARY
           RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
       ) AS ACUM_RANGE
FROM HR.EMPLOYEES
WHERE DEPARTMENT_ID = 90
ORDER BY SALARY;
```

UNBOUNDED PRECEDING = desde el primer registro. CURRENT ROW = hasta el registro/valor actual.

ROWS: el marco avanza fila por fila. RANGE: el marco se basa en el valor del ORDER BY; los empates pueden recibir el mismo acumulado.

En el apunte original aparecía 'BETWEN'; la palabra correcta es BETWEEN.

14. Ejercicios adicionales resueltos

A. Categorizar salarios

Mostrar apellido, salario y categoría BAJO, MEDIO o ALTO.

```
SELECT LAST_NAME,
       SALARY,
       CASE
           WHEN SALARY < 5000 THEN 'BAJO'
           WHEN SALARY BETWEEN 5000 AND 10000 THEN 'MEDIO'
           ELSE 'ALTO'
       END AS CATEGORIA
FROM HR.EMPLOYEES;
```

B. Departamentos que sí tienen empleados

```
SELECT D.DEPARTMENT_NAME
FROM HR.DEPARTMENTS D
WHERE EXISTS (
    SELECT 1
    FROM HR.EMPLOYEES E
    WHERE E.DEPARTMENT_ID = D.DEPARTMENT_ID
);
```

EXISTS solo necesita saber si existe al menos una fila; por eso SELECT 1 es suficiente.

C. Empleados con comisión

```
SELECT LAST_NAME,
       COMMISSION_PCT
FROM HR.EMPLOYEES
WHERE COMMISSION_PCT IS NOT NULL
ORDER BY COMMISSION_PCT DESC;
```

D. Cantidad de empleados por departamento, incluyendo departamentos vacíos

```
SELECT D.DEPARTMENT_NAME,
       COUNT(E.EMPLOYEE_ID) AS CANTIDAD_EMPLEADOS
FROM HR.DEPARTMENTS D
LEFT JOIN HR.EMPLOYEES E
  ON E.DEPARTMENT_ID = D.DEPARTMENT_ID
GROUP BY D.DEPARTMENT_NAME;
```

Aquí COUNT(E.EMPLOYEE_ID) es importante: para un departamento sin empleados, E.EMPLOYEE_ID es NULL y COUNT(columna) devuelve 0. COUNT(*) podría contar la fila generada por el LEFT JOIN.

E. Empleado y manager

```
SELECT E.LAST_NAME AS EMPLEADO,
       M.LAST_NAME AS MANAGER
FROM HR.EMPLOYEES E
LEFT JOIN HR.EMPLOYEES M
  ON E.MANAGER_ID = M.EMPLOYEE_ID;
```

Es un self join: la misma tabla se usa como E y como M.

F. Nómina por departamento

```
SELECT DEPARTMENT_ID,
       SUM(SALARY) AS NOMINA
FROM HR.EMPLOYEES
GROUP BY DEPARTMENT_ID
```

```
ORDER BY NOMINA DESC;
```

G. Posición usando ROW_NUMBER

```
SELECT LAST_NAME,
       SALARY,
       ROW_NUMBER() OVER (ORDER BY SALARY DESC) AS POSICION
FROM HR.EMPLOYEES;
```

15. Chuleta final

BETWEEN → rango inclusivo.

IN → lista de valores.

LIKE → patrones; % = cualquier cantidad, _ = un carácter.

IS NULL → comprobar NULL.

CASE → condiciones flexibles.

DECODE → equivalencias exactas.

COALESCE → primer valor no NULL.

LEFT JOIN → conserva la izquierda.

ON → coincidencias del JOIN; WHERE → filtro final.

COUNT(*) → filas; COUNT(columna) → valores no NULL.

GROUP BY → agrupa; HAVING → filtra grupos.

WITH → CTE; si se referencia a sí misma, puede ser recursiva.

OVER() → función de ventana sin colapsar las filas.

ROWS → marco por filas; RANGE → marco por valores.

EXISTS → comprueba si existe al menos una fila.